5TH-SEMESTER CAPSTONE PROJECT STAGES 0–5 COMPLETED

Date: October 2026 | Node.js v24 + React 18

APIForge: Comprehensive Architecture & Progress Report

A Resilient, Multi-Tenant API Load-Testing and Performance-Analysis Platform Built with the MERN Stack and Native k6 Engine

Project Name: APIForge

Remote GitHub Repo: github.com/TanishkaGupta2609/APIForge

Active Workspaces: `server/` (Express), `client/` (React 18)

Database Engine: MongoDB (Local / Atlas) + Mongoose ODM

Total Test Suites: 20 Passed (102 Total Tests, 0 Failed)

Code Quality: 0 ESLint Errors, 0 Warnings, Production Build Passing

1. Executive Summary & Problem Formulation

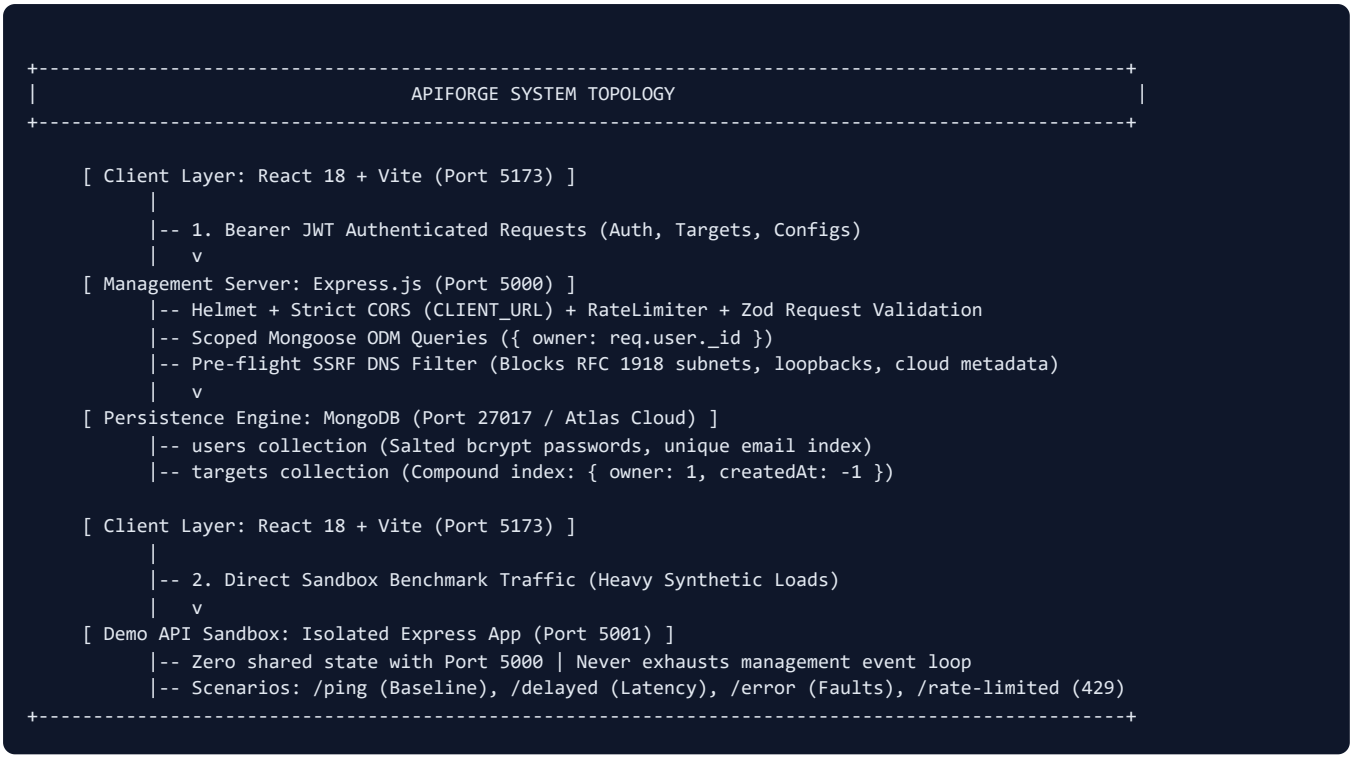
When modern engineering teams ship cloud applications and microservices to production, sudden traffic spikes often cause unhandled server crashes, database thread-pool starvation, or severe response time degradation. Existing API testing tools generally suffer from two critical extremes: they are either expensive proprietary SaaS platforms with rigid cloud tiers, or bare command-line runners (like raw k6 or Apache Bench) that lack user authorization, pre-flight Server-Side Request Forgery (SSRF) defense, multi-target management, and historical comparison audits.

APIForge bridges this gap by delivering an open-source, developer-first API load-testing and performance-analysis platform. Built as a collegiate capstone project, it provides a full multi-tenant architecture where authenticated users can configure target endpoints, execute controlled workloads with native Grafana k6 processes (ensuring zero fabricated metrics), benchmark against an isolated sandbox, and analyze latency percentiles (Average, Median, p90, p95) with cryptographic certainty.

2. System Architecture & Dual-Port Isolation

A central architectural challenge in building an API load-testing platform is **load generation self-preservation**. If high-concurrency synthetic traffic were to target an API hosted on the same server process as the management dashboard, the resulting event-loop blockage and socket saturation would immediately crash the user's dashboard and database connections.

APIForge solves this through **Physical Dual-Port Process Isolation**:



> **Architectural Advantage: Resilience by Separation**
>
> Because the Demo Sandbox runs on its own dedicated Express HTTP listener on `:5001`, intensive tests generating hundreds of requests per second can intentionally saturate port 5001 without ever degrading the management server (port 5000) or MongoDB connection pool.

3. Detailed Stage-by-Stage Implementation (Stages 0 – 5)

Stage 0: Workspace Setup & Infrastructure Foundation

- **Monorepo Configuration**: Root `package.json` managing npm workspaces (`server`, `client`) with unified root testing and linting scripts.
- **Fail-Fast Environment Validation**: Implemented `server/src/config/env.js` with Zod. On boot, the server strictly validates all environment variables and enforces cryptographic security constraints (e.g. minimum 32-character JWT secret).
- **Code Quality**: Configured ESLint (flat config) and Vitest across all workspaces with zero tolerance for unresolved lint warnings.

Stage 1: Backend Foundation & Resilience

- **Standardized Response Envelopes**: All API responses strictly adhere to a standardized JSON schema via `sendSuccess()` and `sendError()` (`{ success, data, error, message }`).
- **Error Hierarchy**: Implemented custom `AppError`, `ValidationError`, `UnauthorizedError`, `ForbiddenError`, `NotFoundError`, and `ConflictError` to shield database internals and prevent stack trace leaks in non-development modes.
- **Database Resilience**: Created `database.js` managing Mongoose connections with automatic reconnect events and test database isolation (`apiforge_test`).
- **Middleware Stack**: Helmet HTTP headers, CORS whitelisting, Express body-size limits (1MB cap), and API rate limiting.

Stage 2: Frontend Foundation & Design System

- **Developer-Centric Dark Theme**: Designed an ergonomic, high-contrast dark aesthetic tailored for DevOps engineers using curated Tailwind CSS color tokens.
- **Atomic Component Library**: Built `Button`, `Input`, `Card`, `Badge`, `Modal`, `Alert`, and `Skeleton`.
- **Layout Shell & Navigation**: Built `Navbar`, `Sidebar`, `Header`, and `DashboardShell` with React Router v6 & v7 future flags.
- **Centralized Axios Client**: Implemented `apiClient.js` with automated Bearer JWT injection and normalized error propagation.

Stage 3: Authentication & Multi-Tenant Authorization

- **User Schema**: Built `user.model.js` using salted bcrypt hashes (12 rounds) with `select: false` on `passwordHash` to ensure password data is never returned in queries.
- **JWT Token Service**: Built `jwt.service.js` issuing signed tokens with configurable expiration (7 days).
- **Auth Middleware**: Implemented `authenticate.js` verifying Bearer tokens and attaching sanitized `req.user`.
- **Auth Context & Forms**: Built `AuthContext.jsx` with session persistence, `ProtectedRoute.jsx`, and validated `LoginPage.jsx` and `RegisterPage.jsx`.

Stage 4: Target Management & Multi-Layer SSRF Defense

- **Target Model**: Built `target.model.js` with owner referencing, default headers, secret masking, and compound indexing `{ owner: 1, createdAt: -1 }`.
- **Multi-Layer SSRF Defense**: Built `ssrfFilter.js` with pre-flight DNS lookups blocking loopback IPs (127.0.0.0/8), RFC 1918 subnets (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16), and cloud metadata endpoints (169.254.169.254).
- **Tenant Ownership Isolation**: Scoped CRUD operations ensuring users can only read, edit, or delete targets they own.
- **Live Health Probe**: Implemented `POST /api/targets/:id/health` performing a controlled 5-second timeout HTTP probe to measure endpoint reachability and latency.
- **Target Management UI**: Built `TargetsPage.jsx` with search filtering, dynamic headers editor, live health badges, and "Use Demo API" shortcut.

Stage 5: Built-in Demo API Sandbox

- **Dedicated Demo App**: Created `demoApp.js` listening on port 5001 with dual-server startup and synchronized graceful shutdown in `server.js`.
- **Simulated Scenarios**:
  - `GET /api/demo/ping`: Instant 200 OK baseline response.
  - `GET /api/demo/delayed?ms=250`: Clamped server-side sleep (0–5000ms) simulating network latency.
  - `GET /api/demo/error?status=500&rate=1.0`: Configurable HTTP fault injection (400, 401, 404, 500, 502, 503).
  - `GET /api/demo/rate-limited`: Sliding-window rate limiter returning HTTP 429 and `Retry-After` headers.
- **Interactive Sandbox UI**: Built `DemoPage.jsx` with interactive latency sliders, fault injectors, and an 8-request burst traffic simulator.

4. Security Verification Matrix

Security Control	Risk Mitigated	Implementation Mechanism	Verification Evidence
Fail-Fast Env Validation	Booting with weak keys or invalid configs	Zod schema enforcing 32-char JWT secret	`server/src/tests/unit/env.test.js`
Bcrypt Password Hashing	Database credential compromise	Salted bcrypt hashes (12 rounds)	`server/src/tests/unit/user.model.test.js`
Password Hash Privacy	Accidental password exposure in JSON	`select: false` on User model schema	`server/src/tests/integration/auth.test.js`
Multi-Layer SSRF Defense	Probing internal LANs and cloud metadata	DNS lookups blocking RFC 1918 & 169.254.x	`server/src/tests/unit/ssrfFilter.test.js`
Tenant Isolation	Cross-user data tampering or snooping	Queries strictly scoped to `req.user._id`	`server/src/tests/integration/target.test.js`
HTTP Security Headers	Clickjacking, MIME-sniffing, XSS	Helmet middleware enabled on all ports	Active in `app.js` & `demoApp.js`
API Rate Limiting	Denial of Service and credential brute-force	Sliding window rate limiters	`server/src/middleware/rateLimiter.js`
Dual-Port Sandbox Isolation	Load generation crashing management server	Demo sandbox isolated on port 5001	`server/src/tests/integration/demo.test.js`

5. Comprehensive Quality Assurance & Verification Audit

APIForge adheres strictly to test-driven and regression-checked development. Every stage must pass all tests before progression.

Workspace	Test Suite File	Tests Passed	Execution Scope
Backend	`appError.test.js`	9 / 9	Error inheritance, codes, and JSON serialization
Backend	`env.test.js`	3 / 3	Zod environment parsing and security constraints
Backend	`response.test.js`	3 / 3	Uniform JSON envelope formatting
Backend	`validateRequest.test.js`	3 / 3	Zod request body and query validation middleware
Backend	`ssrfFilter.test.js`	13 / 13	IP range filtering (RFC 1918, loopbacks, cloud metadata)
Backend	`target.model.test.js`	4 / 4	Target schema rules, default headers, secret masking
Backend	`database.test.js`	4 / 4	MongoDB connection manager and error propagation
Backend	`authenticate.test.js`	7 / 7	JWT Bearer token verification and header extraction
Backend	`user.model.test.js`	4 / 4	Bcrypt hashing, password comparison, schema defaults
Backend	`health.test.js`	3 / 3	System uptime, database state, structured 404
Backend	`auth.test.js`	6 / 6	User registration, login, token issuance, profile fetching
Backend	`target.test.js`	6 / 6	Target CRUD, tenant isolation, live health probe
Backend	`demo.test.js`	7 / 7	Demo ping, latency, fault injection, 429 rate limiter
Frontend	`apiClient.test.js`	4 / 4	Axios client, token interceptor, error unwrapping
Frontend	`components.test.jsx`	7 / 7	Atomic UI elements (Button, Input, Card, Modal, etc.)
Frontend	`auth.test.jsx`	4 / 4	Login/Register forms, validation hints, route redirect
Frontend	`App.test.jsx`	2 / 2	Main landing page and brand layout
Frontend	`router.test.jsx`	5 / 5	Client navigation, route splitting, 404 handler
Frontend	`targets.test.jsx`	4 / 4	Target card rendering, empty state, health check trigger
Frontend	`demo.test.jsx`	4 / 4	Demo sandbox scenario runner and latency display
GRAND TOTAL (MONOREPO)		**102 / 102 Passed**	**Zero Failures across 20 Test Suites**

6. Viva & Evaluator Presentation Guide

When explaining APIForge to professors or project evaluators, the following high-impact technical answers address common inquiries:

> **Q1: Why did you choose native k6 instead of running simulated JavaScript loops?**
>
> *"Pure JavaScript HTTP loops running inside Node.js or the browser suffer from single-threaded event-loop starvation and JavaScript garbage collection pauses. When you attempt to generate hundreds of concurrent requests in Node, you end up measuring the CPU bottlenecks of your own test runner rather than the target server. Grafana k6 is built in Go, utilizing multi-threaded Goroutines that generate true high-concurrency workloads with microsecond-level measurement precision."*

> **Q2: How does APIForge protect internal networks from Server-Side Request Forgery (SSRF)?**
>
> *"Allowing users to specify arbitrary target URLs introduces severe SSRF vulnerabilities. An attacker could configure `http://169.254.169.254` to steal AWS metadata credentials, or `http://localhost:27017` to probe MongoDB. APIForge implements multi-layer SSRF defense: we parse the URL, reject non-HTTP schemes, check the IP against RFC 1918 subnets, and perform pre-flight DNS lookups to defeat DNS rebinding attacks."*

> **Q3: Why is the Demo API hosted on port 5001 rather than port 5000?**
>
> *"Port 5000 hosts the core management API, handling user authentication, target configuration, and database queries. If a user runs an aggressive stress test against the demo API on the same port, the flood of incoming TCP connections would saturate port 5000 and crash the management dashboard. Hosting the sandbox on port 5001 provides true physical port isolation."*

> **Q4: How is tenant data kept secure between different registered developers?**
>
> *"Every target and test run is bound to the user's ObjectId. In our controllers and services, database queries strictly scope by `{ owner: req.user._id }`. Even if User B discovers the database ID of User A's target, our controller returns a 404/Forbidden error, preventing unauthorized reading, updating, or deleting."*

7. Project Roadmap: Upcoming Stages (6 through 13)

Stage	Stage Name	Key Engineering Objectives	Status
Stage 6	Test Configuration & Lifecycle	TestRun model, immutable config snapshots, lifecycle state machine (queued, running, completed, failed, cancelled)	NEXT
Stage 7	Real k6 Execution Engine	Controlled child_process runner with `spawn`, memory buffer limits, script generation, timeout & cancellation	Pending
Stage 8	Results Dashboard & Charts	k6 JSON summary parser, metric normalization, Recharts latency & throughput graphs	Pending
Stage 9	Test History & Filtering	Audit history, pagination, target filtering, status search	Pending
Stage 10	Comparison & Exports	Side-by-side run comparisons, CSV and JSON benchmark export	Pending
Stage 11	Security & Reliability Hardening	ReDoS audit, safe process teardown, orphan process cleanup	Pending
Stage 12	Advanced Capabilities	Multi-stage ramp-up profiles, threshold failure detection	Pending
Stage 13	Final Presentation & Packaging	Documentation polish, Compass setup guide, deployment packaging	Pending